Parameterized INT8 Systolic Matrix-Multiply Accelerator on the Intel Cyclone 10 GX FPGA:

Design, Verification, Synthesis, and In-System Hardware Validation

Boris Gordeev

Department of Electrical and Computer Engineering, Northeastern University

gordeev.b@northeastern.edu · July 2026 · github.com/bgordeev/int8-systolic-matmul-fpga

Keywords: FPGA acceleration · systolic array · INT8 quantization · matrix multiplication · hardware–software co-design

Abstract

Matrix multiplication is a major compute kernel in many neural-network inference workloads, motivating dedicated hardware accelerators. This work presents a fully parameterized INT8 matrix-multiply accelerator implemented in SystemVerilog and realized on the Intel Cyclone 10 GX FPGA (10CX220YF780E5G). Two interchangeable engines share a common interface and are compared directly: a single-MAC sequential baseline and an output-stationary N×N systolic array. For 4×4 INT8 matrices, the systolic engine completes a multiplication in 11 clock cycles versus 81 for the baseline, a 7.4× cycle-count speedup. Correctness is verified bit-exactly against a software golden model in simulation, by a direct cross-check between the two engines, and in-system on physical hardware through an on-chip self-test and a Signal Tap logic-analyzer capture. Post-fit synthesis in Intel Quartus Prime Pro 26.1 at the slow 900 mV, 0°C corner shows the systolic engine uses 822 ALMs and 16 DSP blocks at a restricted Fmax of 165.8 MHz, compared with 506 ALMs and 1 DSP block at 162.6 MHz for the baseline. The area cost of 16× the multipliers and 1.6× the ALMs buys the 7.4× latency reduction, confirming the expected O(N) time and $O(N^2)$ area tradeoff of systolic dataflow. RTL simulation is reproducible with open-source tools. FPGA synthesis, timing analysis, and on-chip validation require Intel Quartus Prime Pro and the target development kit.

Contents

1. Introduction

Matrix multiplication (general matrix–matrix multiply, GEMM) is among the most expensive operations in many deep-learning inference workloads: fully-connected layers, convolutions lowered through im2col, and transformer attention and MLP blocks all reduce to it. Consequently, purpose-built accelerators, such as Google's Tensor Processing Unit [2], GPU tensor cores, and FPGA overlays, are at their core large grids of multiply–accumulate (MAC) units. Understanding and building such hardware is foundational to machine-learning systems research [4].

Two further trends shape this design. First, INT8 quantization is widely used in inference accelerators because representing weights and activations as signed 8-bit integers reduces memory bandwidth and multiplier cost relative to FP32, often with small accuracy loss when the network is calibrated for low precision [3]. Second, FPGAs make the architecture/area/latency design space directly measurable, which is precisely the methodology of computer-architecture research.

This report makes the following contributions: (i) a fully parameterized INT8 GEMM accelerator in SystemVerilog with two interchangeable engines (a sequential MAC baseline and an output-stationary systolic array) sharing a common interface; (ii) a verification flow against a reproducible software golden model, including a bit-exact cross-check between the two engines; (iii) a quantified area-latency tradeoff study on the Cyclone 10 GX, with per-engine post-fit resource and timing data; and (iv) in-system validation on physical hardware, including an on-chip self-test and a logic-analyzer capture confirming the simulated latency on silicon.

Concretely, this report answers three questions. First, what latency improvement does an output-stationary systolic array deliver over a sequential MAC baseline for 4×4 INT8 matrices on this device? Second, what resource cost does that improvement carry in ALMs and DSP blocks? Third, do the simulated results reproduce on physical silicon?

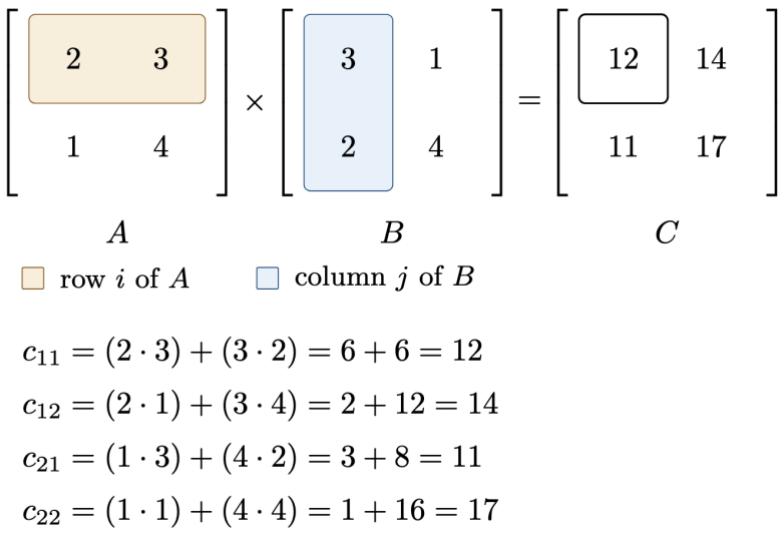
2. Background

2.1 INT8 arithmetic and quantization

Operands are signed 8-bit two's-complement integers (range −128 to +127). A real value x is represented as $x \approx q \cdot s$, where q is the stored integer and s a scale factor fixed at design time. An INT8×INT8 product requires 16 bits, and accumulating K such products requires $16 + \lceil \log_2 K \rceil$ bits. The design therefore uses a 32-bit accumulator (ACC_WIDTH = 32), which precludes overflow for any practical K and removes the need for saturation logic.

2.2 Matrix multiplication and MAC units

For N×N matrices, $C[i][j] = \Sigma_k A[i][k] \cdot B[k][j]$, requiring N^3 MAC operations (counted as $2N^3$ arithmetic operations). The MAC ($acc \leftarrow acc + a \cdot b$) is the atomic unit from which all matrix hardware is composed.



$$c_{11} = (2 \cdot 3) + (3 \cdot 2) = 6 + 6 = 12$$
$$c_{12} = (2 \cdot 1) + (3 \cdot 4) = 2 + 12 = 14$$
$$c_{21} = (1 \cdot 3) + (4 \cdot 2) = 3 + 8 = 11$$
$$c_{22} = (1 \cdot 1) + (4 \cdot 4) = 1 + 16 = 17$$

Figure 1. *Example of 2×2 Matrix Multiplication*

2.3 Systolic arrays

A systolic array is a grid of processing elements (PEs) through which operands flow rhythmically, one neighbor per clock. The concept dates to Kung and Leiserson [1] and underpins the Google TPU [2]. The present design is output-stationary: each PE(i,j) accumulates one result element C[i][j] in place while matrix A streams east and matrix B streams south. Operands must be skewed at the array edges (row i of A delayed i cycles, column j of B delayed j cycles) so that A[i][k] and B[k][j] meet at PE(i,j) on the same cycle. The decisive advantage is operand reuse: each value is fetched once and reused across N PEs, so compute scales as O(N) in time while area scales as O(N^2).

2.4 Target device

The target is the Intel Cyclone 10 GX 10CX220YF780E5G, providing 80,330 ALMs, 192 variable-precision DSP blocks, and M20K embedded memory [5]. The synthesis tool infers DSP blocks from signed multiplication. Although each DSP can pack more than one low-precision multiply, the measured counts in Section 5 correspond one-to-one with the instantiated multipliers at this matrix size.

3. Accelerator Architecture

3.1 Parameters and hierarchy

Table 1. *Top-level design parameters.*

Parameter	Default	Description
DATA_WIDTH	8	Operand width (signed INT8)
ACC_WIDTH	32	Accumulator / result width
MATRIX_SIZE (N)	4	Square matrix dimension

The top level matmul_top instantiates two engines: sequential_matmul (a single shared MAC sequenced by an FSM) and matmul_controller (which wraps an N×N systolic_array of systolic_pe cells). A runtime-select multiplexer chooses which engine drives the shared output.

Both engines present an identical start / busy / done handshake and the same matrix interface, enabling the cross-check of Section 4.

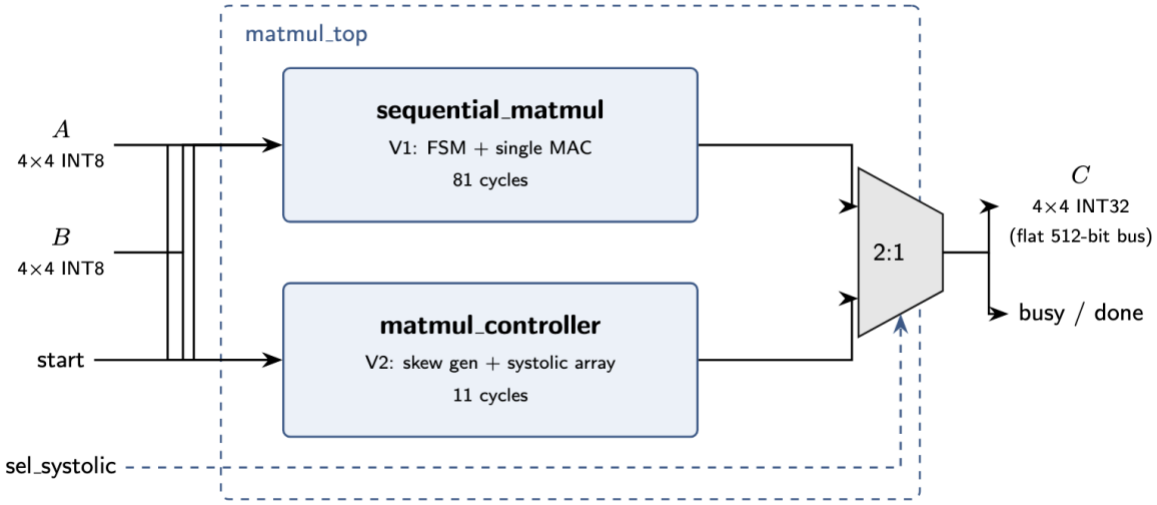


Figure 2. *Top-level block diagram of matmul_top*

3.2 Version 1: sequential MAC baseline

The baseline uses a single MAC driven by a finite state machine that walks the sequence S_IDLE, S_INIT, S_COMPUTE, S_STORE, S_DONE while advancing three nested counters i, j, k. Its latency is 1 (INIT) + N^3 (COMPUTE) + N^2 (STORE) = 81 cycles for N = 4. It serves as a trusted and easily verified reference and as the denominator for speedup.

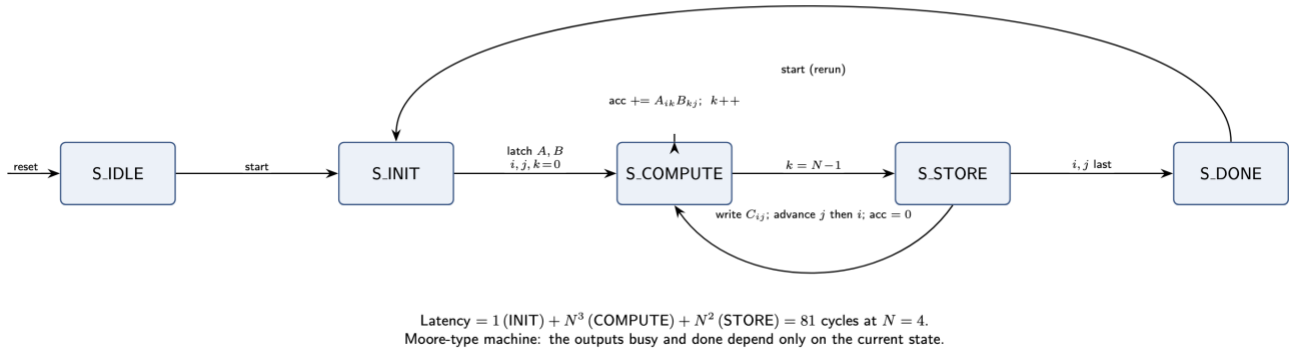


Figure 3. *Sequential Baseline FSM (sequential matmul)*

3.3 Version 2: output-stationary systolic array

The systolic engine instantiates an N×N mesh of PEs. A controller generates the input skew and runs a feed FSM that steps through S_IDLE, S_LOAD, S_FEED, S_DONE. The feed phase is 3N−2 = 10 enabled cycles, and total measured latency is 11 cycles including the S_LOAD cycle. All inter-PE connections are nearest-neighbor, keeping the critical path short as N grows.

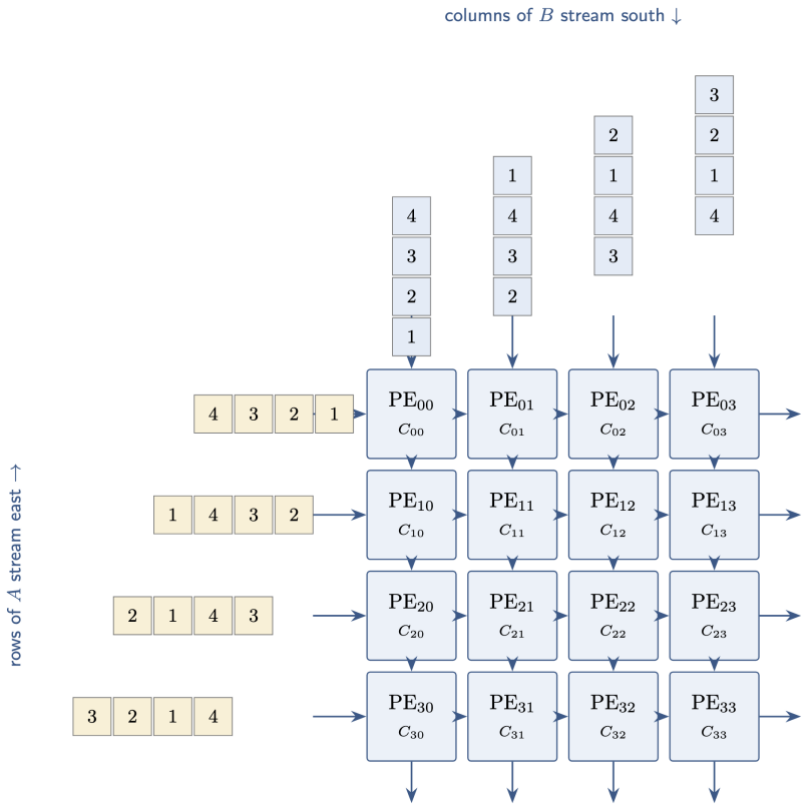


Figure 4. *4×4 Output-Stationary Systolic Array*

4. Verification Methodology and Results

Correctness was established before any hardware step. A Python golden model computes the reference product and emits two's-complement hex vectors with operand magnitudes bounded to preclude overflow. The RTL was exercised with directed tests (small positive, negative, and zero matrices), identity tests, and constrained-random tests. A top-level testbench then runs both engines on identical stimuli and requires bit-exact agreement, after which an independent Python script re-checks the hardware output file against the golden model. All testbenches are self-checking.

Simulation was performed in Questa Intel FPGA Edition. Three self-checking suites were run: the sequential testbench (5 tests), the systolic testbench (7 tests), and the top-level suite (14 checks). All passed. The sequential suite reported 5/5 PASS (Figure 5), the systolic suite 7/7 PASS, and the top-level suite 14/14 PASS. Those 14 checks comprise 5 sequential-engine checks, 5 systolic-engine checks, and 4 engine-agreement comparisons, in which the two engines agreed bit-for-bit on every case (Figure 8). The independent Python re-verification confirmed all 16 result elements (Figure 9). Table 2 lists representative cases from these suites.

Table 2. *Functional verification results (RTL simulation, Questa). Representative cases shown: the full suites comprise 5 sequential, 7 systolic, and 14 top-level cases, all passing.*

Test	Engine	Latency (cyc)	Result
Small positive	Sequential	81	PASS
Negative values	Sequential	81	PASS
Zero matrix	Sequential	81	PASS
Random (hex)	Sequential	81	PASS
Identity × A	Systolic	10 (feed)	PASS
Random (hex)	Systolic	11	PASS
Cross-check (seq vs sys)	**Both**	**81 / 11**	**PASS, bit-exact**
Independent Python re-verify	n/a	n/a	**PASS, 16/16 match**

Summary: the top-level testbench reported 14 passed, 0 failed; both engines produce identical results; and the independent golden-model check confirms all 16 elements.

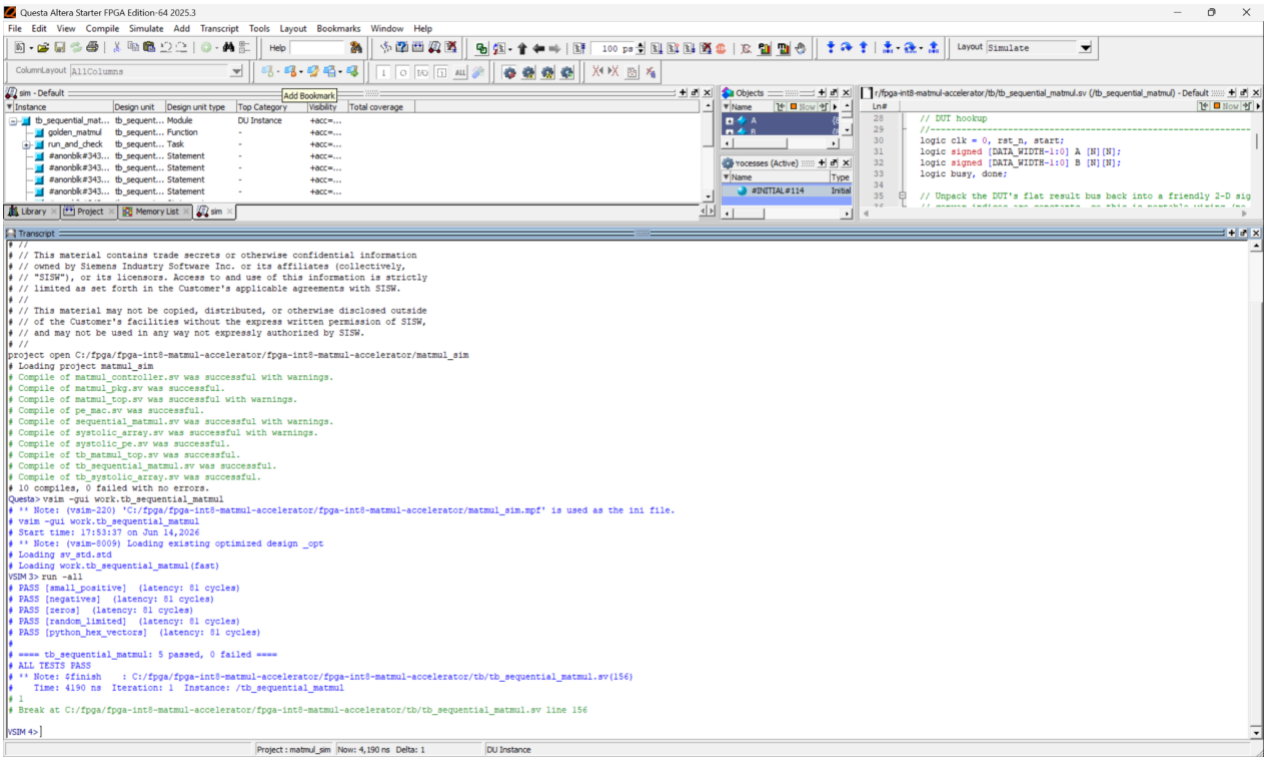


Figure 5. *Sequential baseline self-checking testbench (tb_sequential_matmul). The transcript shows 5/5 PASS with each run reporting 81-cycle latency; this establishes the reference engine is functionally correct before any comparison.*

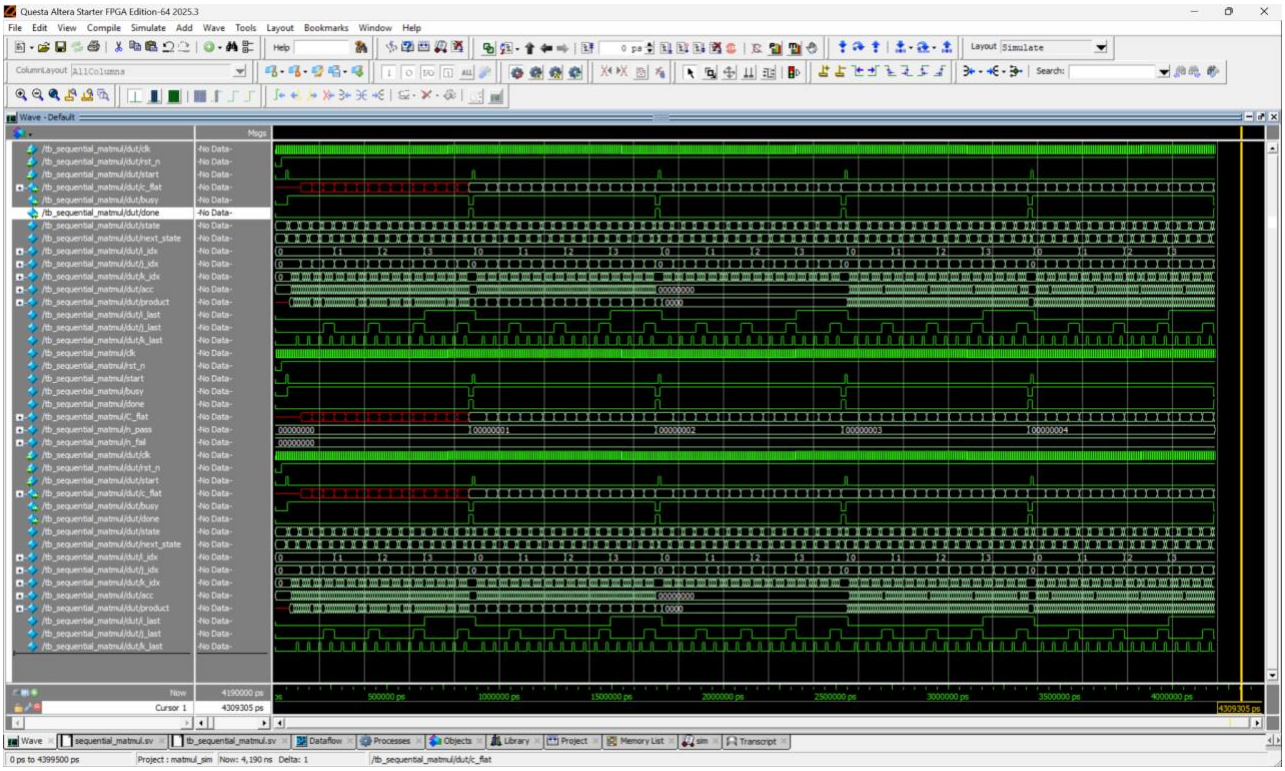


Figure 6. *Sequential baseline waveform. The state signal walks S_IDLE, S_INIT, S_COMPUTE, S_STORE, S_DONE while the accumulator updates once per MAC. This illustrates the single-MAC control flow that yields the 81-cycle latency.*

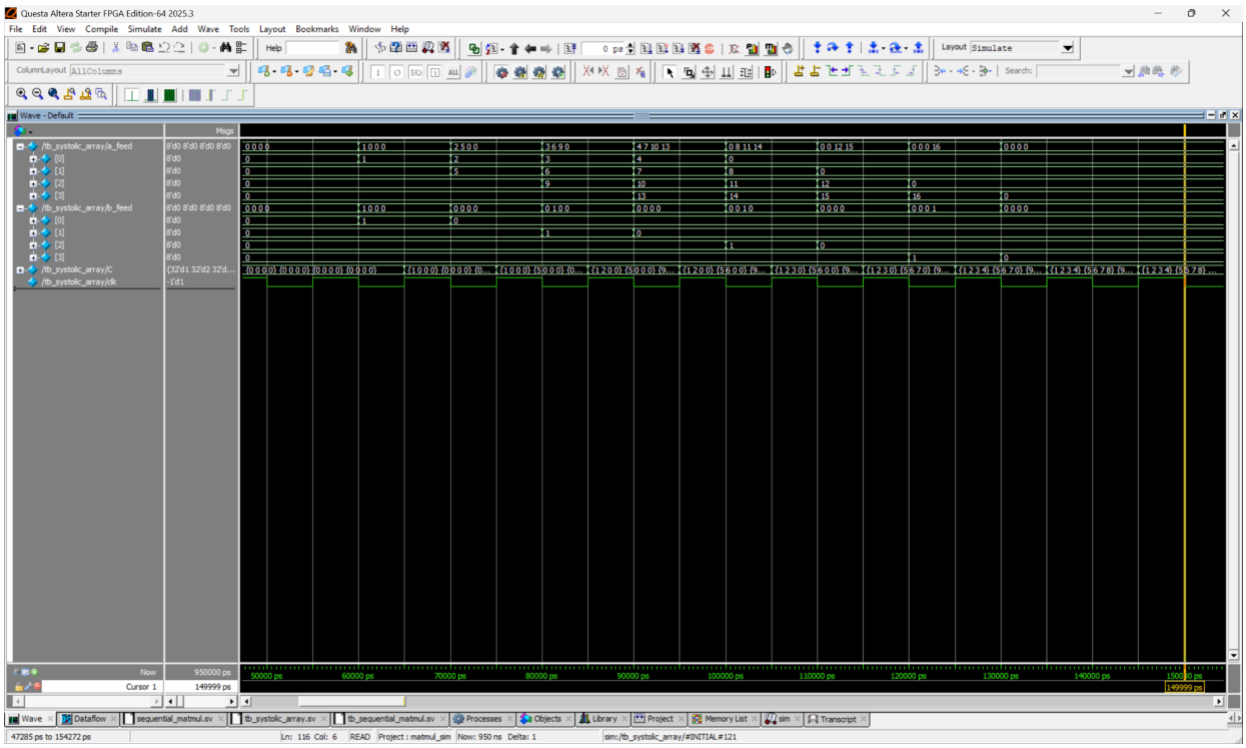


Figure 7. *Systolic engine waveform (decimal radix) for the identity test, where the expected product equals A. The a_feed[0..3] and b_feed[0..3] rows show the one-cycle-per-row input skew, and C fills in to match A, evidencing correct dataflow and skew timing.*

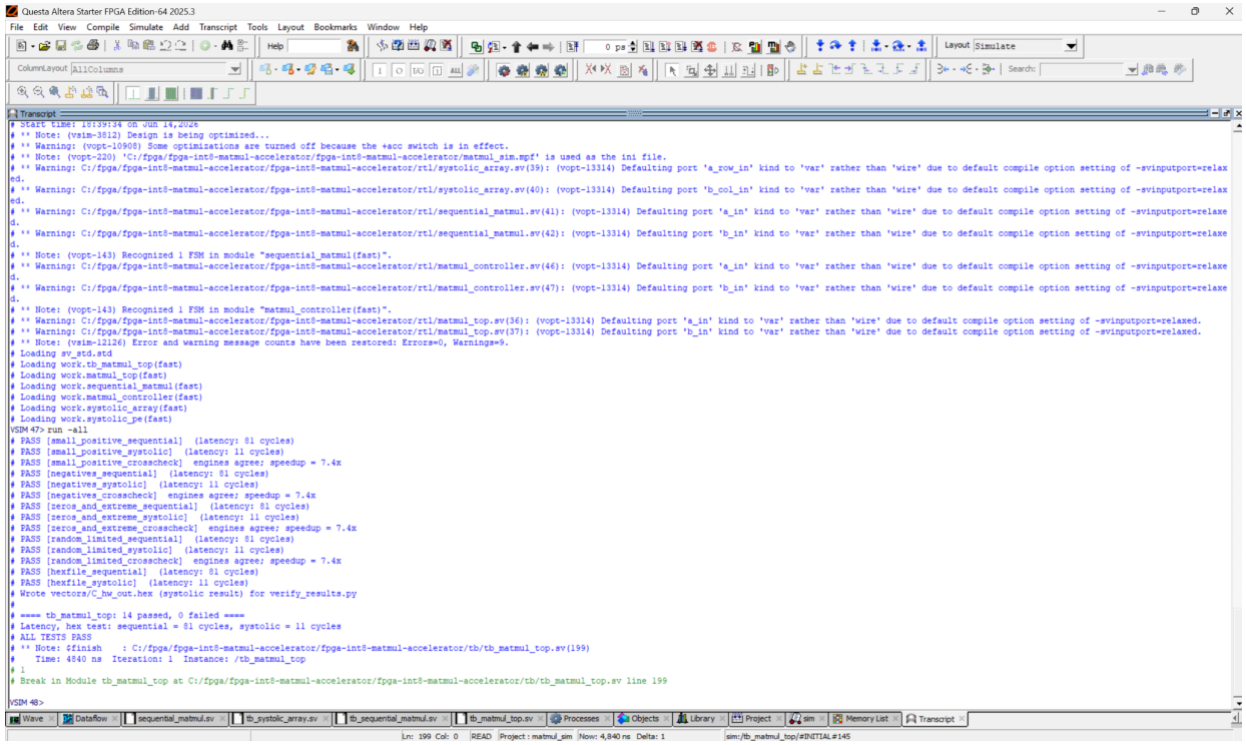


Figure 8. *Top-level cross-check transcript (tb_matmul_top). Both engines run on identical stimuli and agree bit-for-bit on all four engine-agreement checks within the suite's 14 cases; the log reports sequential = 81 and systolic = 11 cycles. This is the primary evidence for the 7.4× cycle-count speedup.*

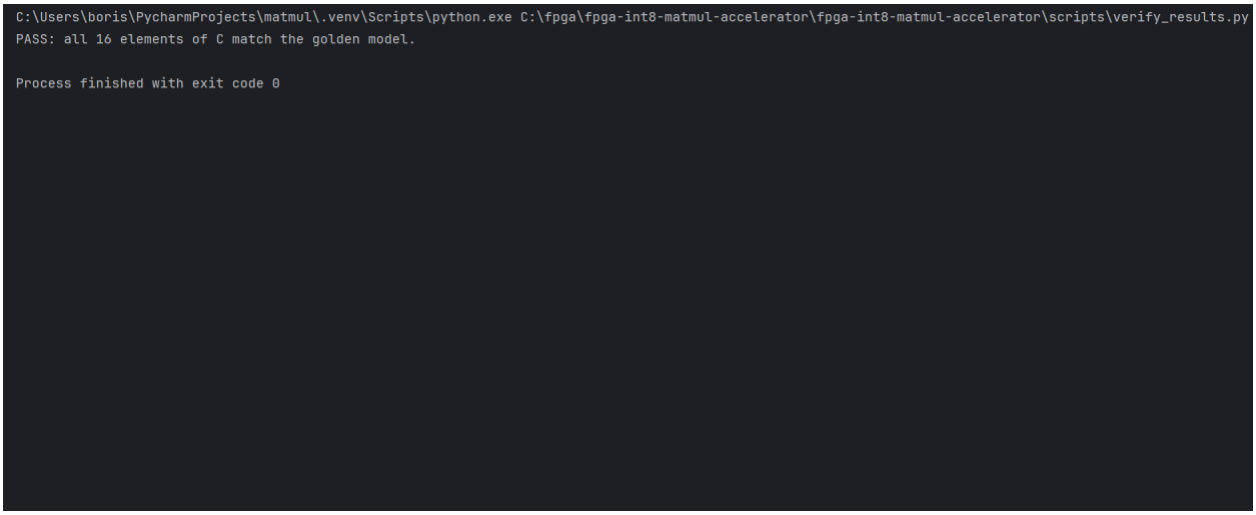


Figure 9. *Independent Python re-verification. A separate script compares the engine output file against the golden model and reports all 16 result elements matching, providing an independent correctness check.*

5. Synthesis Results

Each engine was synthesized separately in Intel Quartus Prime Pro 26.1 targeting 10CX220YF780E5G, with data ports assigned as virtual pins for core characterization and the combined matmul_top was also compiled. Timing is reported at the slow 900 mV, 0°C corner against a 100 MHz constraint. Measured post-fit resource and timing data appear in Table 3. Representative Quartus reports are shown in Figures 10–13.

Table 3. *Post-fit resource utilization and timing (Quartus Prime Pro 26.1, slow 0°C corner, device 10CX220YF780E5G).*

Design	ALMs	Registers	DSP	M20K	Restricted Fmax / Slack
V1 Sequential	506	819	1	0	162.6 MHz / +3.850 ns
V2 Systolic	822	909	16	0	165.8 MHz / +3.969 ns
Combined (matmul_top)	**1,220**	**1,724**	**17**	**0**	**163.2 MHz / +3.871 ns**

All three designs meet the 100 MHz constraint with large positive slack (>3.8 ns) and use no embedded memory. The DSP count tracks the multiplier count exactly: 1 for the single-MAC baseline and 16 for the 4×4 systolic array (one per PE). The combined design (both engines and multiplexer) totals 17 DSP blocks and 1,220 ALMs. Restricted F_{max} exceeds 162 MHz for all configurations, i.e. the design could clock 1.6× above the 100 MHz target.

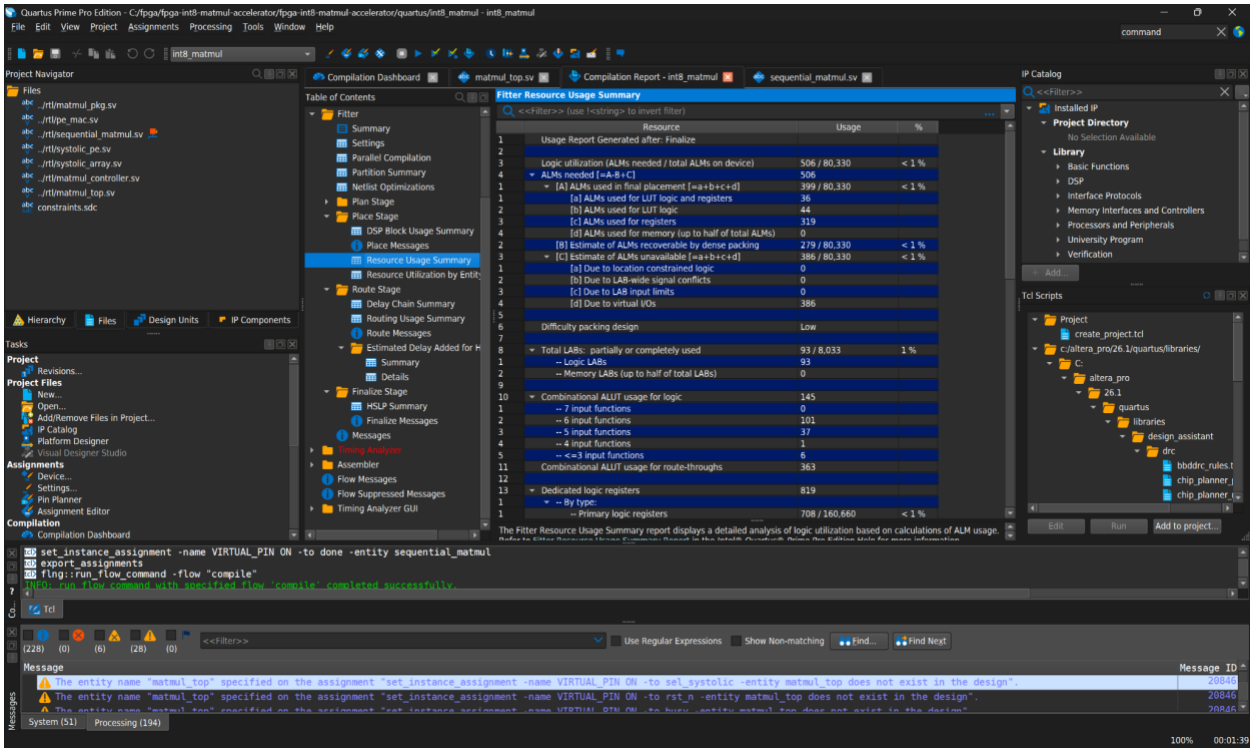


Figure 10. *Quartus Fitter resource usage for the sequential baseline. Confirms a minimal footprint of 506 ALMs and a single DSP block (one multiplier), the area reference against which the systolic engine is compared.*

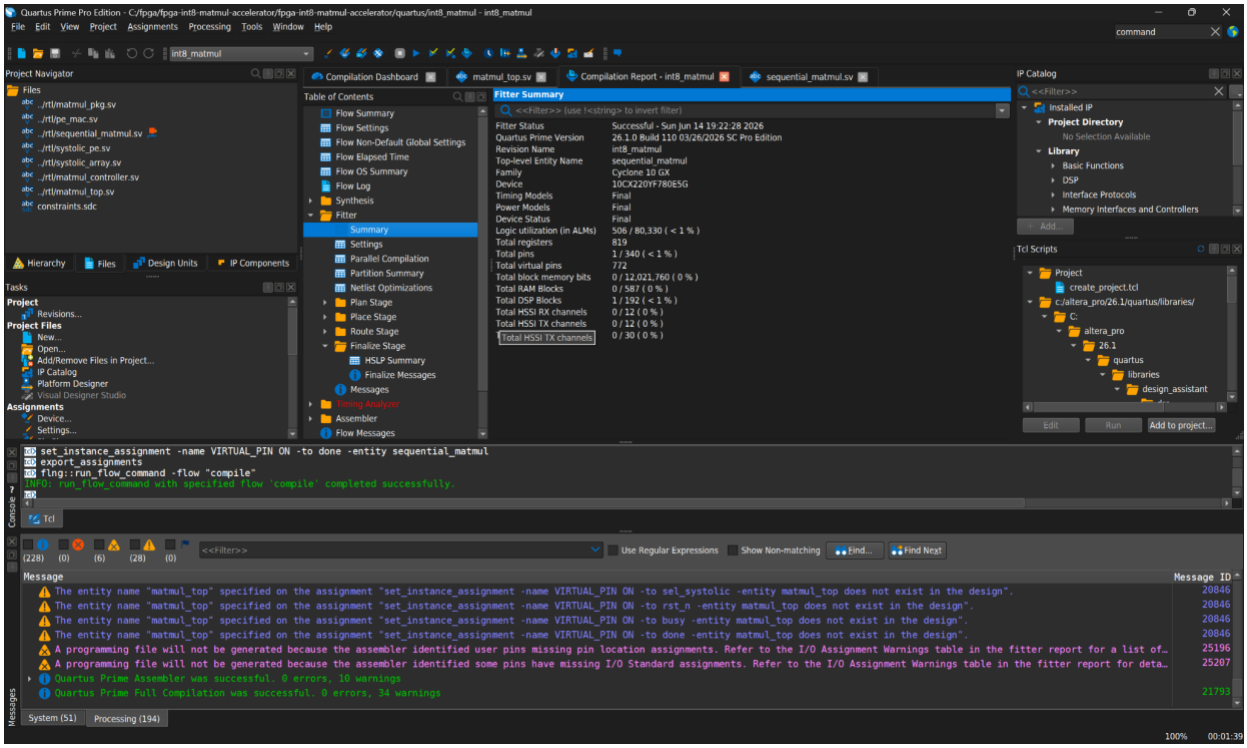


Figure 11. *Flow summary and timing for the sequential baseline. Restricted Fmax of 162.6 MHz confirms the 100 MHz constraint is met with margin.*

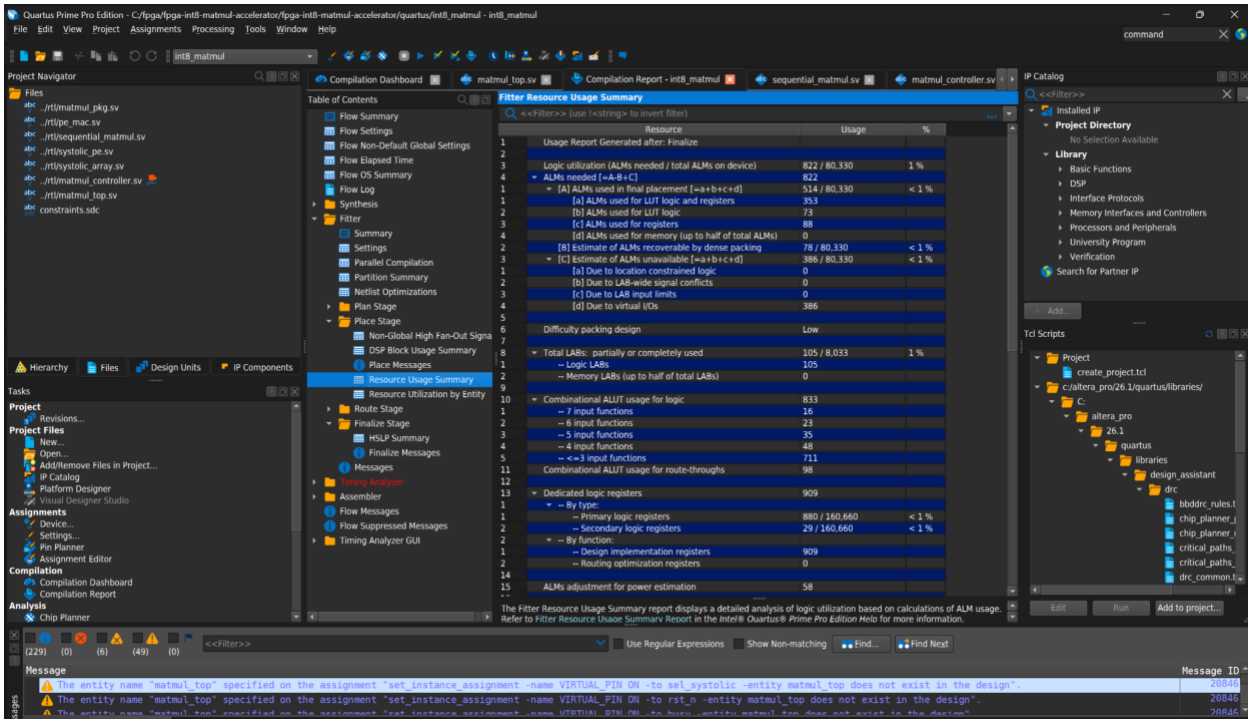


Figure 12. *Quartus Fitter resource usage for the systolic engine. The 16 DSP blocks (one per PE in the 4×4 array) and 822 ALMs quantify the area cost of the parallel architecture.*

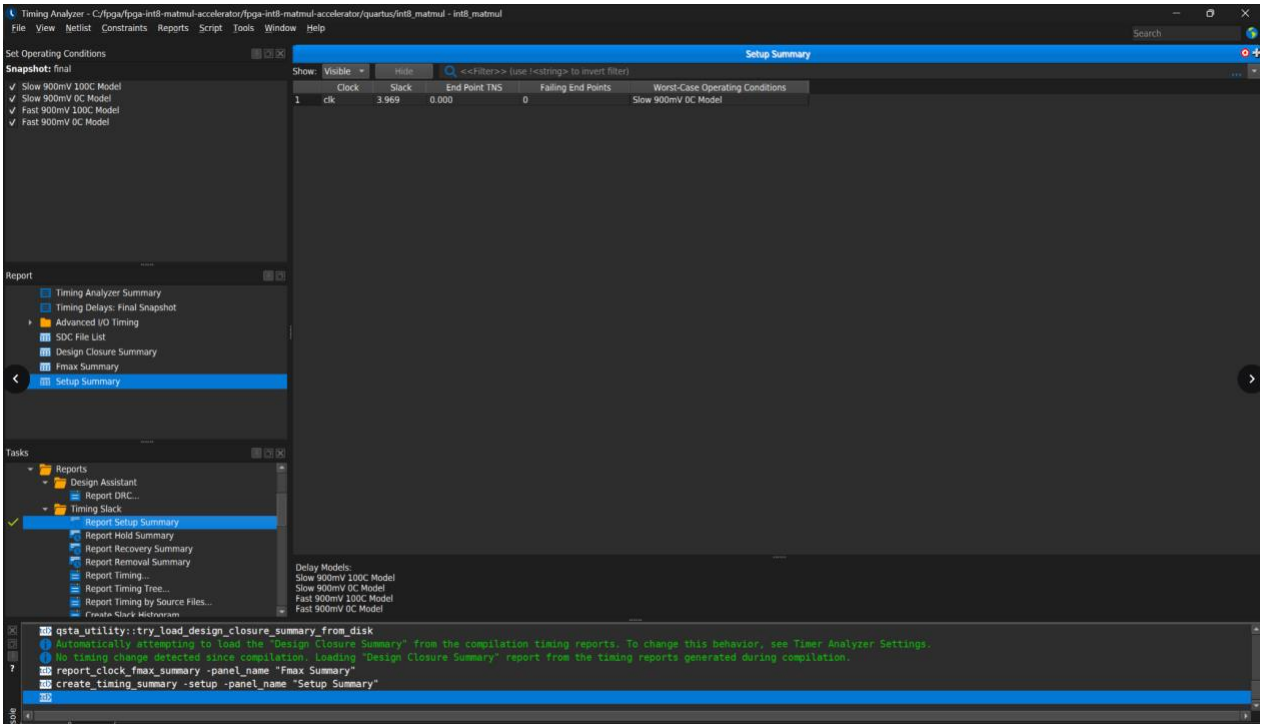


Figure 13. *Setup-timing summary for the systolic engine. Positive worst-case slack of +3.969 ns at the slow corner confirms the design closes timing at 100 MHz (restricted Fmax 165.8 MHz).*

6. Performance Analysis

With N = 4, each multiplication performs $2N^3 = 128$ operations. Throughput is estimated as GOPS = $2N^3 \cdot F_{max}/\text{cycles}/10^9$, evaluated with each engine's own F_{max}. These figures are core-level peak estimates that assume operands are already on-chip and they do not account for memory bandwidth and are not sustained system throughput. The systolic engine sustains 5.82 MACs per cycle versus 0.79 for the baseline. This corresponds to 36 percent utilization of the 16 multipliers, because the fill and drain phases dominate at this small N. Utilization rises toward the 16-MAC peak as N grows or when successive tiles are streamed back-to-back, since the fixed fill and drain cost is then amortized over more compute cycles. Because the systolic engine also closes timing slightly faster, its wall-clock speedup (7.5×) marginally exceeds its cycle speedup (7.4×). Results are summarized in Table 4.

Table 4. *Latency and throughput (per-engine Fmax from Table 3).*

Design	Cycles	Ops	MACs/cyc	Throughput	Speedup
V1 Sequential	81	128	0.79	0.26 GOPS	1.0×
V2 Systolic	**11**	**128**	**5.82**	**1.93 GOPS**	**7.4×**

Key result: the systolic engine completes a 4×4 INT8 matmul in 11 cycles versus 81 for the baseline, a 7.4× cycle-count speedup, verified in simulation, by engine cross-check, and in-system on hardware (Section 8). The speedup scales as $(N^3+N^2+1)/(3N-1)$, increasing with N, so the advantage grows for larger arrays.

7. Resource–Latency Tradeoff

The per-engine data quantify the cost of the speedup. The systolic engine uses 16× the DSP blocks (16 vs 1) and 1.62× the ALMs (822 vs 506) of the baseline to achieve the 7.4× cycle reduction. Normalizing, the speedup-per-ALM-increase is 4.5× (highly area-efficient in logic), while the speedup-per-DSP is 0.46. That is, the design trades abundant DSP resources for latency, which is the intended behavior of a systolic array (N^2 multipliers for O(N) time). On this device the 192 DSP blocks would accommodate a square array of roughly 13×13 before DSP exhaustion (before considering DSP packing of INT8 multiplies), bounding the single-tile array size. Larger matrices would be handled by temporal tiling over a fixed array, as in production accelerators.

Table 5. *Qualitative architecture comparison.*

Design	Main advantage	Main limitation	Best use case
V1 Sequential MAC	Minimal area (1 multiplier); trivially verifiable	$O(N^3)$ latency; control overhead	Golden reference; area-constrained designs
V2 Systolic	O(N) compute time; operand reuse; local wiring scales in Fmax	$O(N^2)$ multipliers; fixed matrix size per build	Throughput-oriented ML inference

8. In-System Hardware Validation

The accelerator was deployed on a physical Intel Cyclone 10 GX FPGA development kit. A self-contained wrapper compiles two fixed INT8 test matrices and their precomputed product into the device. A pushbutton triggers a run, and on-chip logic compares the hardware result against the reference bit-for-bit, asserting a pass indicator on success. Bring-up required resolving the kit's JTAG chain configuration (FMC-JTAG isolation and FPGA-JTAG enable DIP switches) before the 10CX220Y appeared on the chain alongside the system MAX 10.

Result: the on-chip self-test passed on silicon (pass indicator asserted), and a Signal Tap logic-analyzer capture (Figure 14) confirmed the 11-cycle systolic latency measured in simulation, demonstrating that the speedup is reproduced on real hardware, not merely in simulation. The result bus settles to the expected product and result_pass asserts immediately afterward.

Table 6. *Hardware validation summary.*

Metric	Result	Source
Functional self-test (systolic)	PASS (matched reference)	On-board pass indicator
Systolic latency on silicon	11 cycles	Signal Tap capture
Sequential latency	81 cycles	Simulation (tb_matmul_top)
Device verified on JTAG chain	10CX220Y (0x02E120DD)	Quartus Programmer
Result correctness	**result_pass asserted**	**Signal Tap**

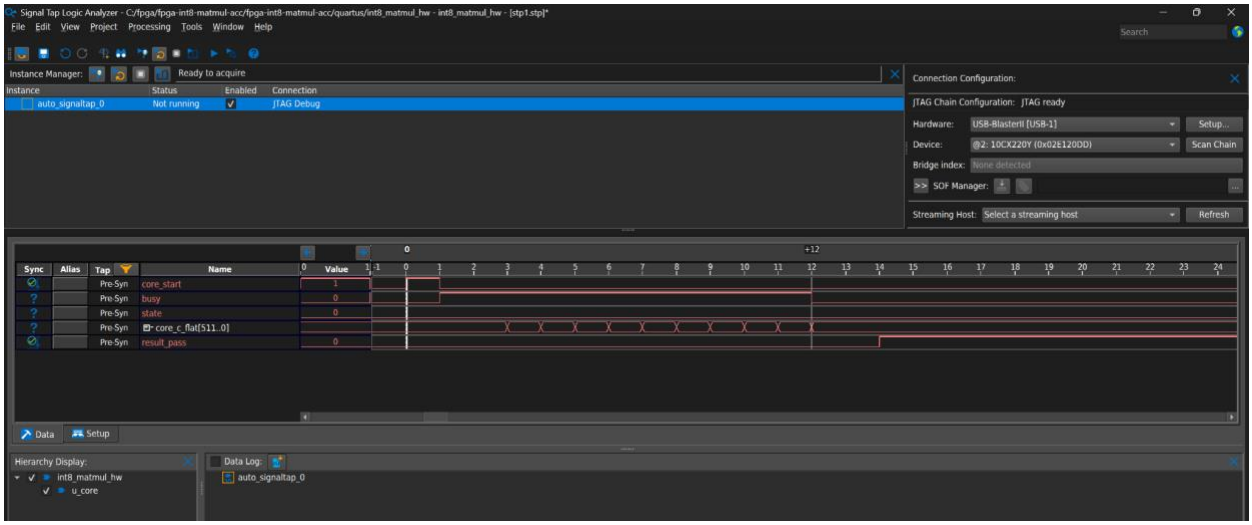


Figure 14. *Signal Tap in-system capture of the systolic engine running on the Cyclone 10 GX. core_start pulses for one cycle at sample 0, busy is asserted for the eleven compute cycles at samples 1–11 and deasserts at sample 12, where core_c_flat settles to the expected product, and result_pass asserts two cycles later at sample 14 once the on-chip comparison stage has registered the match. The 11-cycle busy window reproduces the simulated latency on silicon.*

9. Limitations

- Operands enter and results leave through wide parallel ports (virtual pins in core synthesis) whereas a deployable accelerator would use on-chip RAM with a streaming or memory-mapped interface. The hardware demonstration therefore uses fixed compiled-in operands rather than streamed data.
- Matrix size is fixed at build time to MATRIX_SIZE so there is no run-time tiling for larger matrices.
- No saturation logic is included. The 32-bit accumulator makes overflow impossible for the tested sizes, but extreme parameterizations could overflow silently.
- Reported throughput characterizes the compute core. A full system would also be bounded by memory bandwidth.

10. Future Work

- Larger systolic arrays with a resource and Fmax scaling study, approaching the device DSP limit.
- On-chip M20K double-buffering to overlap operand load with computation.
- An Avalon-MM or AXI4-Lite slave interface and DRAM streaming with tiling for large GEMM.
- Mapping a convolution layer via im2col and an end-to-end INT8 quantized inference demonstration with measured accuracy.
- Integration with a RISC-V soft core as a memory-mapped coprocessor.

11. Conclusion

This work presented a parameterized INT8 matrix-multiply accelerator on the Intel Cyclone 10 GX, comparing a sequential MAC baseline against an output-stationary systolic array. The systolic engine achieves a 7.4× cycle-count speedup (11 vs 81 cycles for 4×4), verified bit-exactly in simulation, by engine cross-check, and, most significantly, in-system on physical hardware with a Signal Tap capture confirming the latency on silicon. Per-engine synthesis quantifies the cost: 16× the DSP blocks and 1.6× the ALMs for the 7.4× latency reduction, a textbook demonstration of the O(N) time / $O(N^2)$ area tradeoff of systolic dataflow. The most promising next step is scaling the array and adding a streaming memory interface to evaluate end-to-end neural-network inference.

Author Contribution

This is an individual project. The author designed and wrote all SystemVerilog RTL, including the parameter package, the MAC, both engines, the systolic processing element and mesh, the skew controller, and the top level. The author also wrote the self-checking testbenches and the Python golden model and verifier, performed all synthesis and timing analysis in Quartus Prime Pro, and carried out the hardware bring-up, including resolving the JTAG chain configuration and capturing the in-system Signal Tap result.

Reproducibility and Availability

Source RTL, testbenches, the Python golden model, Quartus project scripts, and build instructions are available at *https://github.com/bgordeev/int8-systolic-matmul-fpga*. RTL simulation can be reproduced with Icarus Verilog or Questa Intel FPGA Edition. FPGA synthesis, timing closure, and the Signal Tap

hardware validation require Intel Quartus Prime Pro and a Cyclone 10 GX development board, and cannot be reproduced with open-source tools alone. Exact commands are documented in the repository README.

References

[1] H. T. Kung and C. E. Leiserson, “Systolic arrays (for VLSI),” in Sparse Matrix Proceedings, 1979.

[2] N. P. Jouppi et al., “In-datacenter performance analysis of a tensor processing unit,” in Proc. ISCA, 2017.

[3] B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in Proc. CVPR, 2018.

[4] V. Sze, Y.-H. Chen, T.-J. Yang, and J. Emer, “Efficient processing of deep neural networks: A tutorial and survey,” Proc. IEEE, 2017.

[5] Intel Corporation, Cyclone 10 GX Device Overview, C10GX51001.

[6] Intel Corporation, Cyclone 10 GX Variable Precision DSP Blocks User Guide, UG-20112.